Feasible-First Exploration for Constrained ML Deployment Optimization in Crash-Prone Hierarchical Search Spaces

Christian Lysenstøen
Department of Electrical Engineering and Computer Sciences
University of California, Berkeley
christian@lysenstoen.net

April 2026

Abstract

Deploying machine learning models under production constraints requires joint optimization over model family, quantization scheme, runtime backend, and serving configuration. This induces a hierarchical mixed-variable search space in which many configurations are invalid: evaluations may crash, exceed memory limits, or violate latency constraints. Standard black-box optimizers such as Tree-structured Parzen Estimators (TPE) and constrained Bayesian optimization are effective when valid configurations are common, but they can spend a large fraction of a small evaluation budget on invalid or uninformative trials in hostile deployment spaces. This paper studies that regime and asks whether optimization should be decomposed into an explicit exploration stage followed by model-guided exploitation.

We propose *Thermal Budget Annealing* (TBA), a feasible-first exploration procedure that maps valid and feasible regions before warm-starting TPE. The method includes two robustness mechanisms for hostile hardware: *trial timeouts* that abort clearly infeasible evaluations early, and *subspace blacklisting* that temporarily suppresses categorical subspaces after repeated failures. We also introduce *DeployBench*, a benchmark suite for deployment optimization with hierarchical structure, hidden crash zones, hard constraints, and unequal evaluation costs. On synthetic benchmarks and real GPU deployment with five pre-trained vision models across five GPU targets (NVIDIA H100, A100, RTX 5080, L4, and T4), the proposed hybrid improves model-family discovery under tight constraints while reducing wasted budget relative to cold-start TPE. The hybrid discovers the globally optimal model family (`vit_tiny`) in 5/5 seeds on H100, 4/5 on A100 and L4, and 3/5 on T4, compared to TPE's 3/5 on H100, 4/5 on A100, 3/5 on L4, and 3/5 on T4. On the RTX 5080 with 10 seeds, the hybrid discovers `vit_tiny` in 8/10 seeds versus TPE's 3/10. Multi-hardware experiments show that the hybrid matches or exceeds TPE on discovery across the tested hardware settings, with the clearest gains on H100 and RTX 5080. The results support a narrower but practically important conclusion: in partially invalid deployment spaces, early exploration quality determines which model families are available to the exploitation phase.

Keywords: black-box optimization, constrained optimization, deployment optimization, warm-start optimization, hierarchical search, hidden constraints, crash-aware search, AutoML

1 Introduction

Modern ML deployment rarely consists of a single model-selection decision. Practical deployment requires choosing a model family, quantization scheme, runtime backend, batch size, and backend-specific execution parameters under strict constraints on latency, memory, and, in some settings, power. The resulting optimization problem is structurally different from conventional

hyperparameter optimization. First, the search space is *hierarchical*: the activeness of some parameters depends on earlier structural choices. Second, the space is often *partially invalid*: some configurations fail outright because of out-of-memory events, unsupported backend–operator combinations, compiler failures, or quantization incompatibilities. Third, evaluation costs are unequal because exporting, compiling, and benchmarking different configurations can vary from seconds to minutes.

These characteristics matter most when the evaluation budget is small. A method that spends its first several trials on invalid configurations may never observe the model family that would have produced the best feasible deployment. This issue is especially acute for model-guided optimizers that rely on early samples to shape later exploitation. If early observations are narrow, then later sampling may reinforce that narrow view. The central question in this paper is therefore not whether a new optimizer can dominate established methods in all settings, but whether a *feasible-first exploration stage* improves performance in deployment spaces that are crash-heavy, hierarchical, and budget-constrained.

This question is timely for two reasons. First, recent work in hyperparameter optimization and constrained optimization has emphasized hierarchical spaces, hidden constraints, and warm-starting as first-class concerns rather than secondary engineering details [Akiba et al., 2019, Awad et al., 2021, Lindauer et al., 2022, Watanabe and Hutter, 2023, Tian et al., 2024]. Second, deployment optimization itself has become a systems problem: practical inference quality depends on the interaction between model architecture, quantization, runtime, and hardware target, not on model choice alone [Chen et al., 2018, Reddi et al., 2020, Zaharia et al., 2024, Agrawal et al., 2024].

The proposed method is simple. Phase 1 uses crash-aware simulated annealing to search for feasible regions and diversify model-family discovery. Phase 2 warm-starts Optuna's TPE sampler from the Phase 1 history. Two additional robustness mechanisms address hostile hardware environments: trial timeouts abort evaluations whose early latency measurements already exceed 5× the constraint threshold, and subspace blacklisting temporarily suppresses categorical values (e.g., a specific quantization mode) after three consecutive failures. The resulting method, TBA→TPE, is not presented as a replacement for TPE in benign spaces. The claim is narrower: when invalidity is high and budgets are small, explicit feasible-first exploration can improve model-family discovery and reduce wasted budget enough to make model-guided exploitation more effective.

Contributions.

1. We formalize budget-constrained deployment optimization in partially invalid hierarchical spaces and identify *premature exploitation* as a central failure mode of model-guided search in crash-heavy regimes.

2. We propose TBA, a feasible-first exploration procedure with adaptive temperature control, crash-aware state transitions, trial timeouts, subspace blacklisting, and an adaptive handoff into TPE.

3. We introduce *DeployBench*, a benchmark suite for constrained deployment search with hidden crash regions, hierarchical conditionals, and unequal evaluation costs.

4. We show on synthetic tasks and real GPU deployment across five hardware targets that warm-starting TPE with feasible-first exploration improves discovery of the best model family under tight constraints while reducing wasted budget relative to cold-start TPE. The hybrid matches or exceeds TPE on discovery in the tested hardware settings, with the clearest gains on H100 (5/5 vs 3/5) and RTX 5080 (8/10 vs 3/10).

2 Background and Problem Setting

Deployment optimization differs from conventional hyperparameter optimization in ways that are operationally significant. In training-time HPO, a trial typically returns a scalar objective together with optional soft constraints [Hutter et al., 2019]. In deployment optimization, an evaluation may fail to return any useful objective at all due to runtime incompatibilities, export failures, out-of-memory conditions, or compiler exceptions. This creates a *partially invalid* search problem in which optimization must reason simultaneously about objective quality, constraint satisfaction, and evaluability.

A second distinction is that deployment search spaces are naturally *hierarchical*. Structural decisions such as backend selection activate or deactivate lower-level parameters. For example, selecting `torch_compile` introduces compiler-related options that are irrelevant under eager execution, while selecting `onnxruntime` activates export and execution-provider choices. Consequently, the dimensionality of the active search space depends on the configuration itself.

A third distinction is that deployment trials have highly unequal cost. A configuration that runs in eager mode on a small model may complete in seconds, whereas a configuration that triggers graph export, compilation, or repeated failed allocation attempts may consume substantially more wall-clock time before yielding either a result or an exception. This cost asymmetry means that wasted evaluations are not merely statistically inefficient; they are operationally expensive.

These observations motivate a decomposition of the optimization process into two conceptual tasks: *region discovery* and *within-region optimization*. Region discovery concerns identifying model families and system configurations that can be evaluated successfully and satisfy constraints. Within-region optimization concerns improving the deployment objective after such regions have been identified. The central premise of this work is that standard model-guided optimizers conflate these tasks too early in crash-heavy settings.

3 Related Work

3.1 Hyperparameter Optimization

The field of hyperparameter optimization has matured substantially over the past decade. Random search [Bergstra and Bengio, 2012] established that uniform sampling can outperform grid search in high-dimensional spaces. Bayesian optimization with Gaussian processes [Snoek et al., 2012, Garnett, 2023] introduced model-guided sequential search, and Tree-structured Parzen Estimators [Bergstra et al., 2011] provided a scalable alternative for conditional spaces. Multi-fidelity methods such as Hyperband [Li et al., 2017] and BOHB [Falkner et al., 2018] reduce cost by truncating unpromising trials, while DEHB [Awad et al., 2021] combines Hyperband with evolutionary search for improved scalability. Optuna [Akiba et al., 2019] and SMAC3 [Lindauer et al., 2022] provide practical frameworks with support for conditional search spaces, and the recent c-TPE extension [Watanabe and Hutter, 2023] adds native constraint-aware sampling to the TPE family. Existing HPO benchmarks [Eggensperger et al., 2021, Pfisterer et al., 2022] have standardized evaluation methodology but generally assume that most configurations are evaluable.

These methods are designed for spaces where the vast majority of configurations produce valid results. The deployment setting studied here, in which 30–80% of random configurations may crash or violate constraints, lies outside those assumptions.

3.2 Constrained and Hidden-Constraint Optimization

Constrained Bayesian optimization has been studied through probability-of-feasibility weighting [Gardner et al., 2014], unknown-constraint modeling [Gelbart et al., 2014], and information-

based acquisition strategies [Hernández-Lobato et al., 2016]. More recently, optimization under hidden or partially observed constraints has received renewed attention [Tian et al., 2024]. The present work connects that perspective to ML deployment, where crashes are not merely noisy evaluations but an intrinsic feature of the search space.

3.3 ML Systems Optimization

Deployment optimization for ML systems has gained attention through compiler-level approaches such as TVM [Chen et al., 2018] and Ansor [Zheng et al., 2020], which automate operator scheduling. TensorRT and ONNX Runtime provide backend-specific runtime optimization. MLPerf [Reddi et al., 2020] has established the importance of inference benchmarking under realistic latency constraints. More recent systems work has examined edge deployment [Hymel et al., 2023], large-scale inference simulation [Agrawal et al., 2024], and the broader shift toward compound AI systems where deployment configuration spans multiple interacting components [Zaharia et al., 2024]. The problem considered here differs from this literature by treating deployment configuration itself as a constrained search problem over model family, quantization mode, backend, and serving parameters.

3.4 Simulated Annealing and Hybrid Methods

Simulated annealing [Kirkpatrick et al., 1983] is a classical metaheuristic for combinatorial optimization. Adaptive temperature schedules and reheating strategies are well studied [Ingber, 1989]. Evolutionary strategies [Hansen and Ostermeier, 2001] provide an alternative paradigm for black-box search in mixed spaces. The contribution of the present work is not a new simulated annealing theory, but the use of feasible-first annealing as an exploration front-end for model-guided optimization in crash-heavy deployment spaces, augmented with robustness mechanisms that address the specific failure modes of hostile hardware environments.

4 Problem Formalization

4.1 Hierarchical Deployment Search Space

Definition 4.1 (Hierarchical mixed-variable search space)**.** *A hierarchical mixed-variable search space $\mathcal{X}$ is defined by variables $\{x_1, \ldots, x_d\}$ where each variable has a type $\tau_i \in \{\text{categorical}, \text{integer}, \text{continuous}\}$, a domain $\mathcal{D}_i$, and an optional activation condition $\phi_i : \mathcal{X} \to \{0, 1\}$. A configuration $\boldsymbol{x} \in \mathcal{X}$ assigns values only to active variables. The active set $\mathcal{A}(\boldsymbol{x}) = \{i : \phi_i(\boldsymbol{x}) = 1\}$ depends on the configuration itself.*

In the deployment setting studied here,

$$\mathcal{X} = \mathcal{M} \times \mathcal{Q} \times \mathcal{B} \times \mathcal{S} \times \prod_{i \in \mathcal{A}(\boldsymbol{x})} \mathcal{D}_i, \tag{1}$$

where $\mathcal{M} = \{\texttt{resnet18}, \texttt{resnet50}, \texttt{mobilenet_v2}, \texttt{efficientnet_b0}, \texttt{vit_tiny}\}$, $\mathcal{Q} = \{\texttt{fp32}, \texttt{fp16}, \texttt{int8_dynami}$
$\mathcal{B} = \{\texttt{pytorch_eager}, \texttt{torch_compile}, \texttt{onnxruntime}\}$, and $\mathcal{S} = \{1, 2, 4, 8, 16, 32\}$.

4.2 Constraints and Partial Invalidity

The deployment optimization problem is:

$$\begin{aligned} \boldsymbol{x}^* = \arg\max_{\boldsymbol{x} \in \mathcal{X}} \quad & f(\boldsymbol{x}) \\ \text{subject to} \quad & g_j(\boldsymbol{x}) \le c_j, \quad j = 1, \ldots, J, \\ & \boldsymbol{x} \in \mathcal{X}_{\text{valid}} \subseteq \mathcal{X}. \end{aligned} \tag{2}$$

Here f denotes a deployment objective such as accuracy or throughput, g_j are constraint functions such as latency$_{p95}$ or peak memory, and $\mathcal{X}_{\text{valid}}$ denotes the unknown subset where evaluation does not crash.

Total constraint violation is:

$$V(\boldsymbol{x}) = \sum_{j=1}^{J} \max\left(0,\ g_j(\boldsymbol{x}) - c_j\right). \tag{3}$$

A configuration is feasible if $V(\boldsymbol{x}) = 0$ and $\boldsymbol{x} \in \mathcal{X}_{\text{valid}}$.

Definition 4.2 (Crash zone)**.** *The crash zone $\mathcal{X}_{crash} \subset \mathcal{X}$ is the subset of configurations for which evaluation fails to produce a valid result. The crash zone is not known a priori and must be discovered through evaluation.*

The effective search space decomposes as

$$\mathcal{X} = \underbrace{\mathcal{X}_{\text{crash}}}_{\text{crashed}} \cup \underbrace{(\mathcal{X}_{\text{valid}} \setminus \mathcal{C})}_{\text{valid but infeasible}} \cup \underbrace{\mathcal{F}}_{\text{feasible}}, \tag{4}$$

where $\mathcal{C} = \{\boldsymbol{x} : g_j(\boldsymbol{x}) \le c_j,\ \forall j\}$ and $\mathcal{F} = \mathcal{X}_{\text{valid}} \cap \mathcal{C}$.

Definition 4.3 (Budget-constrained deployment optimization)**.** *Given $\mathcal{X}$, objective f, constraints $\{(g_j, c_j)\}_{j=1}^{J}$, and evaluation budget $B \in \mathbb{N}$, the task is to find the best feasible configuration using at most B evaluations, where each evaluation may crash and has variable cost $t(\boldsymbol{x}) > 0$.*

Two metrics capture optimizer performance. The first is simple regret:

$$r(n) = f(\boldsymbol{x}^*_{\text{global}}) - \max_{i \le n,\ \boldsymbol{x}_i \in \mathcal{F}} f(\boldsymbol{x}_i). \tag{5}$$

The second is wasted budget fraction:

$$W(n) = \frac{|\{i \le n : \boldsymbol{x}_i \notin \mathcal{F}\}|}{n}. \tag{6}$$

4.3 A Heuristic Characterization of Premature Exploitation

The central failure mode considered in this work is *premature exploitation*. In a crash-heavy space, an optimizer that transitions into model-guided sampling after only a small number of successful evaluations may have observed only a narrow subset of model families. Subsequent exploitation then reinforces that biased sample.

Remark 4.1 (Probabilistic intuition for premature exploitation)**.** *If the probability of sampling a specific model family in one trial is $p_m \approx 1/K$ for K families, and the survival probability is $(1 - \rho_{crash})$, then the probability of discovering that family in n_0 random trials is*

$$\mathbb{P}(\textit{discover family } m \textit{ in } n_0 \textit{ trials}) = 1 - \left(1 - p_m(1 - \rho_{crash})\right)^{n_0}. \tag{7}$$

For $K = 5$, $\rho_{crash} = 0.6$, and $n_0 = 10$, this yields approximately 0.57, which predicts that only about 4–6 of 10 seeds would discover a given family during the initial exploration stage. This is directionally consistent with the observed behavior of cold-start TPE in the edge-tight setting (Section 9).

5 Research Hypotheses

The proposed method is motivated by three hypotheses.

H1: Early structural coverage matters. In hierarchical deployment spaces, the main risk of early optimization is not merely poor local refinement but insufficient structural coverage. If early evaluations fail to touch promising model families or backend regimes, later model-guided search is conditioned on a biased subset of the space.

H2: Invalidity changes the exploration–exploitation trade-off. In standard black-box optimization, random or quasi-random early trials are often adequate to initialize a surrogate. In partially invalid spaces, the same number of trials may produce too few successful observations to support reliable model-guided exploitation.

H3: Warm-started exploitation is preferable to cold-start exploitation in hostile spaces. If an exploration phase can deliberately accumulate a more diverse set of valid and feasible observations, then a subsequent model-guided optimizer should perform better than if it had spent the same budget on unguided early exploration.

These hypotheses define the scientific scope of the paper. The contribution is therefore not framed as a universally superior optimizer, but as evidence for a specific architectural claim: in crash-heavy constrained deployment spaces, explicit feasible-first exploration improves the quality of subsequent model-guided search.

6 Method: Thermal Budget Annealing

6.1 Overview

TBA→TPE consists of two phases. Phase 1 uses simulated annealing to search for feasible regions while deliberately traversing structural choices. Phase 2 injects the full history from Phase 1 into a constrained TPE study and uses TPE for model-guided exploitation. Two additional robustness mechanisms—trial timeouts and subspace blacklisting—reduce wasted budget on hostile hardware by aborting clearly doomed trials and avoiding repeatedly failing categorical subspaces. The full optimizer is summarized in Algorithm 1.

6.2 Phase 1: Feasible-First Simulated Annealing

Phase 1 operates with a two-stage objective. Before any feasible configuration has been found, the optimizer minimizes total violation $V(\boldsymbol{x})$. Once a feasible configuration is found, the objective switches to maximizing $f(\boldsymbol{x})$ over feasible states.

The temperature follows an adaptive schedule with reheating:

$$T_{n+1} = \begin{cases} \alpha_s T_n & \text{if improved,} \\ T_n & \text{if not improved and patience not exceeded,} \\ \min(\beta T_n,\ \gamma T_0) & \text{if patience exceeded.} \end{cases} \tag{8}$$

In the reported experiments, $\alpha_s \in \{0.92, 0.95\}$ with slower cooling during the feasibility stage to maintain broader exploration, $\beta = 2.5$, $\gamma = 0.8$, and patience is 4 consecutive non-improving trials.

6.3 Acceptance Criteria

The acceptance rule depends on the current stage.

Feasibility stage. A new state is accepted deterministically if it reduces total violation. Otherwise, it is accepted with probability

$$\mathbb{P}(\text{accept}) = \exp\left(-\frac{V(\boldsymbol{x}') - V(\boldsymbol{x}_{\text{curr}})}{T_n \cdot \sigma_V}\right), \tag{9}$$

where

$$\sigma_V = \max\left(0.1,\ \frac{1}{J}\sum_{j=1}^{J} |c_j| \cdot 0.1\right). \tag{10}$$

Optimization stage. If the proposal is infeasible, it is rejected outright. If it improves the objective, it is accepted. Otherwise, it is accepted with probability

$$\mathbb{P}(\text{accept}) = \exp\left(-\frac{f(\boldsymbol{x}_{\text{curr}}) - f(\boldsymbol{x}')}{T_n \cdot \sigma_f}\right), \tag{11}$$

where

$$\sigma_f = \max\left(0.01,\ \frac{f_{\max} - f_{\min}}{2}\right). \tag{12}$$

6.4 Hierarchical Neighbor Proposals

Neighborhood proposals must respect the hierarchical structure of the space. With probability $p_s(n)$, the optimizer performs a structural move by changing a categorical variable such as model family, backend, or quantization mode. Conditional variables activated by the new structure are sampled from their valid domains; deactivated variables are removed. With probability $1-p_s(n)$, the optimizer performs a numerical move by perturbing an active integer or continuous parameter.

The structural mutation probability decays linearly over the budget:

$$p_s(n) = 0.5 - 0.35 \cdot \frac{n - n_{\text{init}}}{B - n_{\text{init}}}. \tag{13}$$

This schedule reflects the intuition that early search should privilege structural exploration while later search should privilege local refinement.

6.5 Crash Handling and Exploration Mechanisms

Crashed configurations are never accepted as the current state. However, they are retained in history and therefore contribute to the optimizer's implicit map of invalid regions. This design allows the search to learn crash boundaries without accumulating in crash-prone states.

Two additional mechanisms stabilize exploration. *Elite restart* replaces the current state with the 2nd or 3rd best feasible configuration every 12 trials during the first 70% of budget. *Decaying snap-back* returns to the global best feasible state with probability $0.4(1 - n/B)$ after accepting a worse feasible move.

6.6 Trial Timeouts

On hostile hardware such as the T4, certain configurations (e.g., `int8_dynamic` quantization) produce latencies of 500ms or more—25× the 20ms constraint. Running these trials to completion wastes both wall-clock time and evaluation budget. Trial timeouts address this by aborting evaluations early when they are clearly infeasible.

After the initial warmup iterations, the profiler measures the average latency over the first few timed iterations. If this average exceeds $\tau = 5 \cdot c_{\text{latency}}$ (i.e., 100ms for a 20ms constraint), the

trial is immediately aborted and marked as early-stopped. The factor of 5× ensures that only *catastrophically* infeasible trials are terminated; borderline-infeasible trials still run to completion and provide useful constraint information.

This mechanism is applied at the benchmark level and therefore benefits *all* optimizers equally. In the reported experiments, trial timeouts saved 7–47 seconds per seed on the T4 and 6–25 seconds per seed on the H100, with the savings concentrated on `int8_dynamic` and large-batch configurations.

6.7 Subspace Blacklisting

A more targeted mechanism addresses the *exploration trap* identified in early T4 experiments, where the SA phase repeatedly proposed configurations from a catastrophically infeasible categorical subspace (e.g., `quantization=int8_dynamic`) without learning to avoid it.

The subspace tracker maintains a per-value failure counter for each categorical variable. When a categorical value accumulates three consecutive crash or infeasible outcomes, it is temporarily *blacklisted*: the SA neighbor proposal masks that value from the categorical domain. Three safeguards prevent over-restriction:

1. *Cooldown*: after 8 trials, a blacklisted value is automatically restored to the active domain.
2. *Success reset*: any successful trial using a blacklisted value immediately removes the blacklist.
3. *Safety valve*: the tracker never blacklists *all* values of any categorical variable; at least one value always remains available.

Subspace blacklisting is applied only during Phase 1 (SA exploration). Phase 2 (TPE exploitation) receives the full, unrestricted search space. This design reflects the intent that blacklisting is an exploration-efficiency mechanism, not a permanent constraint on the search.

6.8 Adaptive Handoff into TPE

The handoff from exploration to exploitation occurs when the optimizer has observed sufficient evidence to initialize TPE reliably:

$$\text{handoff if } n \geq n_{\min}, \quad N_{\text{feas}} \geq 5, \quad N_{\text{bad}} \geq 3, \tag{14}$$

with a hard maximum of $n_{\max} = 15$ trials. Here N_{feas} is the number of feasible observations and N_{bad} is the number of crashed or infeasible observations. This criterion ensures that the TPE phase receives both positive and negative information about the space.

6.9 Phase 2: Warm-Started TPE

All Phase 1 evaluations are injected into an Optuna study as completed trials, including crashed and infeasible configurations together with constraint metadata. The constrained TPE acquisition can be expressed schematically as

$$\text{EI}_c(\boldsymbol{x}) = \frac{\ell(\boldsymbol{x})}{g(\boldsymbol{x})} \cdot \mathbb{P}(\boldsymbol{x} \in \mathcal{C} \mid \mathcal{H}), \tag{15}$$

where $\ell(\boldsymbol{x})$ and $g(\boldsymbol{x})$ are density estimates over good and bad observations, respectively. The crucial point is that Phase 2 begins from a broader and more informative set of initial observations than cold-start TPE would obtain from unguided early trials.

Table 1: Deployment search space variables.

Variable	Type	Domain	Condit
`model_name`	Categorical	{resnet18, resnet50, mobilenet_v2, efficientnet_b0, vit_tiny}	No
`backend`	Categorical	{pytorch_eager, torch_compile, onnxruntime}	No
`quantization`	Categorical	{fp32, fp16, int8_dynamic}	No
`batch_size`	Integer	$\{1, 2, 4, 8, 16, 32\}$	No
`num_threads`	Integer	$[1, 8]$	backend $\neq$ `tor`

6.10 Computational Cost

The practical cost of deployment optimization is determined jointly by trial count and wall-clock time. Let $t(\boldsymbol{x})$ denote the time required to evaluate configuration $\boldsymbol{x}$. Then cumulative wall-clock cost is

$$T_{\text{wall}}(n) = \sum_{i=1}^{n} t(\boldsymbol{x}_i). \tag{16}$$

Trial timeouts directly reduce T_{wall} by aborting catastrophically infeasible evaluations before completion. Subspace blacklisting indirectly reduces it by steering proposals away from subspaces where $t(\boldsymbol{x})$ is high and outcomes are consistently infeasible.

7 DeployBench: A Benchmark for Crash-Heavy Deployment Optimization

7.1 Design Principles

Existing HPO benchmarks [Eggensperger et al., 2021, Pfisterer et al., 2022] assume that most configurations produce valid results. DeployBench is designed to reflect deployment regimes where this assumption fails:

1. many random configurations are invalid (30–80% invalidity),
2. the space is hierarchical and mixed-variable,
3. constraints are hard rather than softly penalized,
4. evaluation costs vary substantially,
5. each trial is logged in JSONL format for reproducible behavioral analysis.

7.2 Synthetic Benchmarks

Two synthetic tasks were used during development and evaluation. *Crashy Branin* augments the standard Branin function with a categorical mode variable, an integer resolution variable, explicit crash zones, and feasibility constraints, yielding approximately 65% combined invalidity. *Hierarchical Rosenbrock* activates different subsets of continuous variables according to a categorical mode, thereby varying effective dimensionality across the space.

7.3 Real Deployment Benchmark

The real benchmark evaluates deployment configurations on ImageNette [Howard, 2020] using five pretrained vision models, three backends, three quantization modes, six batch sizes, and a conditional thread parameter. The resulting search space is summarized in Table 1.

Table 2: Deployment scenarios.

Scenario	Constraints	Objective	Budget
`edge_tight`	latency$_{p95} \leq$ 20ms, memory $\leq$ 512MB	maximize accuracy	25
`server_throughput`	latency$_{p99} \leq$ 100ms, memory $\leq$ 4096MB	maximize throughput	25

Before accounting for conditional inactivity, this space contains 2160 structural combinations. Known crash patterns include `onnxruntime` with various model–precision combinations, `int8_dynamic` producing extreme latencies on slow GPUs, and large model–batch size combinations causing GPU OOM. These patterns are intentionally not filtered from the search space.

Each evaluation follows a standardized procedure: model loading, optional quantization, backend compilation or export, 30 warmup iterations, 100 timed iterations (with trial timeout check after warmup), resource measurement, accuracy evaluation on 500 ImageNette validation images, and explicit memory cleanup. All evaluations are wrapped in exception handling so that a crash produces a logged result rather than terminating the experiment.

8 Experimental Setup

8.1 Baselines

Three baselines are evaluated under identical search spaces, budgets, and seeds:

1. **Random Search** [Bergstra and Bengio, 2012]: uniform random sampling, reporting the best feasible result.
2. **Optuna TPE** [Akiba et al., 2019, Watanabe and Hutter, 2023]: constrained tree-structured Parzen estimation with native conditional handling.
3. **TBA (pure SA)**: the feasible-first exploration stage without TPE handoff. This ablation isolates the contribution of warm-starting.

All optimizers share the same benchmark infrastructure, including trial timeouts. Subspace blacklisting is applied only within the TBA and TBA→TPE optimizers during Phase 1.

8.2 Metrics

Performance is assessed along four dimensions:

1. best feasible objective (mean ± std across seeds),
2. wasted budget fraction $W(B)$,
3. time to first feasible configuration,
4. model-family discovery rate.

8.3 Hardware, Software, and Protocol

Primary experiments were conducted on an NVIDIA RTX 5080 Laptop GPU (16 GB GDDR7) with an Intel Core Ultra 9 275HX and 16 GB DDR5 RAM. Multi-hardware generalization experiments (Section 9.4) were conducted on four additional GPU targets via Google Colab: NVIDIA H100 80GB, A100 40GB, L4 24GB, and T4 16GB. Software versions: PyTorch 2.6, torchvision 0.21, ONNX Runtime 1.21, Optuna 4.2, Python 3.12. All primary experiments use 10 random seeds (0–9); multi-hardware experiments use 5 seeds (0–4). All experiments across

Table 3: Crashy Branin: best feasible objective (higher is better) and wasted budget fraction. Bold indicates best objective at each budget.

	TBA→TPE		TBA (SA)		TPE		Random	
B	Obj	W	Obj	W	Obj	W	Obj	W
10	−5.83	34%	**−5.65**	40%	−7.29	54%	−8.44	65%
15	−5.54	25%	−4.71	32%	**−2.59**	42%	−6.37	66%
20	**−3.18**	30%	−4.62	27%	−4.19	40%	−5.17	63%
25	−2.29	28%	−3.68	28%	**−2.14**	35%	−4.12	64%
30	−2.05	30%	−2.93	27%	**−1.97**	31%	−4.41	67%
40	−1.57	26%	−3.95	24%	**−1.38**	30%	−2.98	63%
50	−1.45	23%	−2.66	20%	**−1.15**	29%	−2.56	66%

Table 4: Ablation on Crashy Branin at $B = 30$.

Variant	**Best Objective**	**Wasted %**
TBA full (SA + all components)	-2.93 ± 2.37	27%
w/o no-late-restarts	-3.62 ± 2.58	30%
w/o decaying snap-back	-3.45 ± 2.71	29%
w/o adaptive p_s	-3.21 ± 2.50	28%
TBA→TPE hybrid	$\mathbf{-2.05 \pm 0.71}$	30%

all GPUs use the same codebase (v2-improvements branch, commit `7ea93e2`). Conditional parameters are exposed consistently across methods, and crashed trials are recorded rather than silently filtered.

9 Results

9.1 Synthetic Benchmarks: Budget Sweep

Table 3 reports results on Crashy Branin across budgets from 10 to 50.

Three findings emerge from Table 3. First, at $B = 20$, the hybrid outperforms all baselines including TPE (−3.18 vs −4.19), indicating that warm-starting with crash-aware exploration data can outperform cold-start model-guided search when the budget is tight. Second, at $B =$ 25–30, the hybrid and TPE achieve comparable objectives, but the hybrid exhibits lower variance (± 0.79 vs ± 1.33 at $B = 25$), indicating more consistent performance. Third, the wasted budget fraction for the hybrid (23–34%) is consistently lower than TPE's (29–54%), with the largest gap at small budgets. This pattern is consistent with H2 and H3: the main gain comes from better early exploration rather than stronger asymptotic exploitation.

9.2 Ablation

The ablation indicates that restricting restarts to the first 70% of budget is the most impactful component among the pure-TBA modifications, while the TPE handoff provides the largest overall gain over pure SA with substantially reduced variance.

9.3 Real Deployment: Edge-Tight Scenario (RTX 5080)

Table 5 reports results under tight edge deployment constraints on the primary hardware target.

The aggregate accuracies are close, and the hybrid's edge-tight mean accuracy (0.761) is essentially tied with random search (0.762). The hybrid's advantage is not a large mean-accuracy

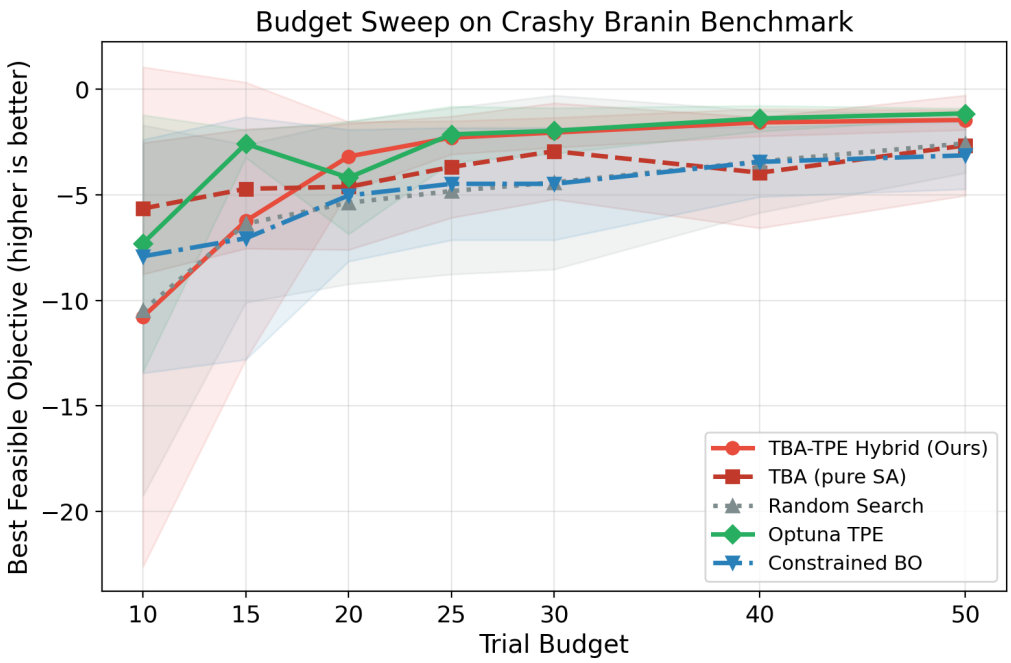

Figure 1: Best feasible objective versus evaluation budget on Crashy Branin (10 seeds, mean ± std). The hybrid outperforms cold-start TPE at $B = 20$ and tracks it closely at higher budgets, while pure SA and random search lag behind.

Table 5: Edge-tight deployment on RTX 5080: latency$_{p95}$ $\leq$ 20ms, memory $\leq$ 512MB, budget=25, 10 seeds.

Optimizer	**Accuracy (mean ± std)**	**Wasted %**	**Feasible seeds**
Random Search	0.762 ± 0.008	74%	10/10
TBA→TPE Hybrid	0.761 ± 0.010	42%	10/10
TBA (pure SA)	0.752 ± 0.019	30%	10/10
Optuna TPE	0.752 ± 0.010	36%	10/10

gain but rather improved discovery behavior and budget efficiency (Figure 3). The more informative result is model-family discovery.

Table 6 shows that cold-start TPE discovers `vit_tiny`, the best constrained model family in this setting (76.6% accuracy), in only 3 out of 10 seeds. In the remaining seeds, TPE converges to `resnet50` (accuracy 0.746) or `efficientnet_b0` (accuracy 0.742) and exploits them without evaluating `vit_tiny`. This is the premature exploitation phenomenon described in Remark 4.1.

This is the central empirical finding. The hybrid avoids the `resnet50` trap because Phase 1 explores broadly enough to encounter `vit_tiny` before the exploitation phase narrows the search. Random search also discovers `vit_tiny` frequently, but it wastes 74% of its budget on invalid trials, nearly double the hybrid's 42%. Our primary practical objective is not universal objective dominance, but improved structural discovery under tight budgets with lower wasted evaluation cost.

The per-seed model discovery table (Table 7) reveals the behavioral difference in detail. TPE converges to `resnet50` in 7 of 10 seeds without ever discovering `vit_tiny`, while the hybrid's structural exploration during Phase 1 reaches `vit_tiny` in 8 of 10 seeds before exploitation begins.

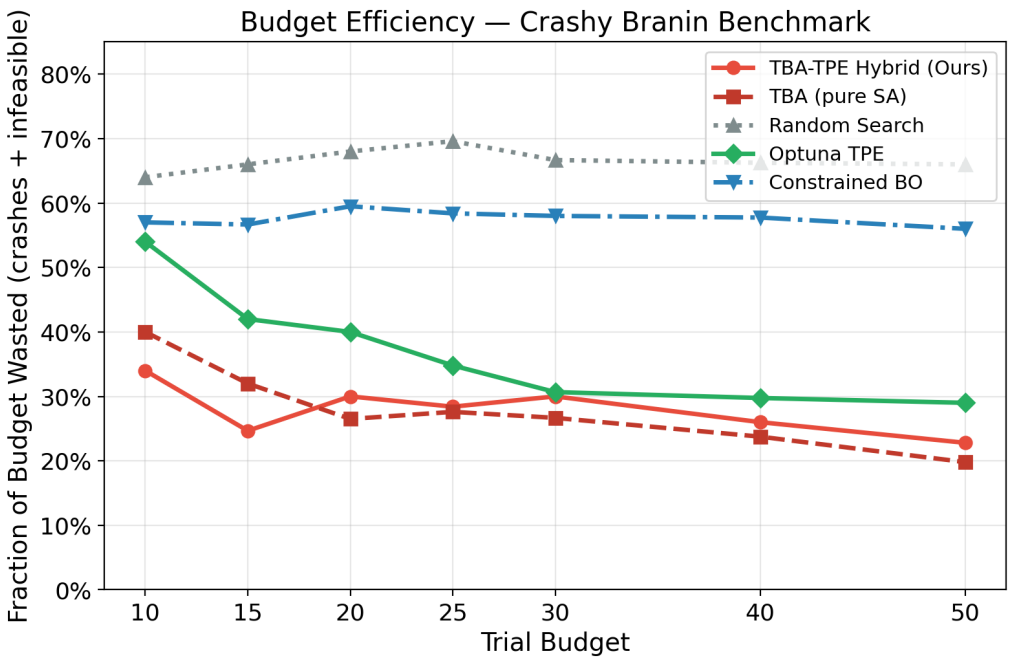

Figure 2: Wasted budget fraction versus evaluation budget on Crashy Branin. The hybrid consistently wastes less budget than TPE across all budgets tested. Random search wastes 63–67% regardless of budget.

Table 6: Seeds (out of 10) discovering `vit_tiny`, the globally best architecture under edge-tight constraints on the RTX 5080.

Optimizer	Discovery rate
TBA→TPE Hybrid	**8 / 10**
Random Search	8 / 10
TBA (pure SA)	6 / 10
Optuna TPE	3 / 10

9.4 Multi-Hardware Generalization

To test whether the hybrid's advantage generalizes across hardware targets with different search-space characteristics, we repeated the edge-tight experiment on four additional GPUs: NVIDIA H100, A100, L4, and T4 (5 seeds each). Each GPU induces a different *effective hostility level* because hardware speed determines which configurations satisfy the 20ms latency constraint. Faster GPUs admit more feasible configurations, reducing the crash-and-infeasibility rate; slower GPUs tighten the feasibility boundary.

Table 8 reports the `vit_tiny` discovery rate across all five GPUs. Table 9 reports mean accuracy and wasted budget fraction.

Four findings emerge from the multi-hardware experiments. Figure 5 visualizes the central trade-off: the hybrid (dark blue) achieves discovery rates comparable to random search (gray) while wasting far less budget, and it never falls below TPE (red) on discovery despite maintaining comparable or lower waste.

The hybrid matches or exceeds TPE on discovery. The hybrid discovers `vit_tiny` at a rate equal to or higher than TPE on all five hardware targets: 5/5 vs 3/5 on H100, 4/5 vs 4/5 on A100, 8/10 vs 3/10 on RTX 5080, 4/5 vs 3/5 on L4, and 3/5 vs 3/5 on T4. The gains are

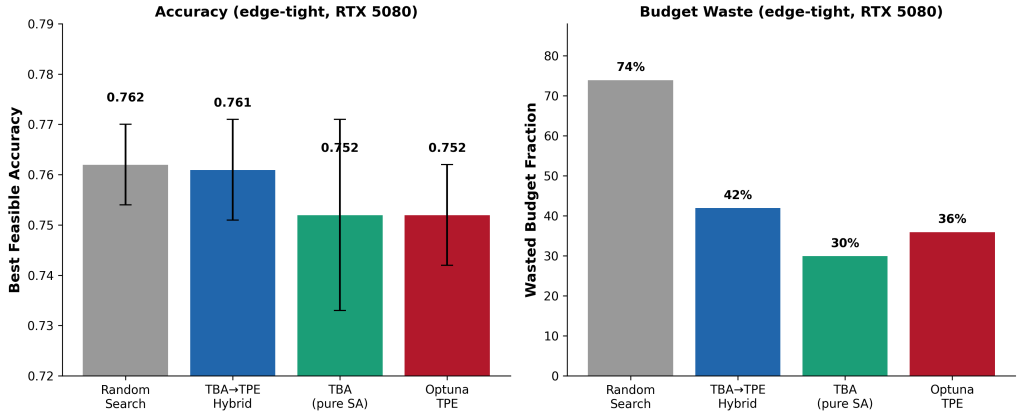

Figure 3: Edge-tight deployment results on RTX 5080: accuracy and wasted budget fraction by optimizer. The hybrid achieves accuracy comparable to random search while wasting roughly half the budget.

Table 7: Best model family discovered per seed under edge-tight constraints on RTX 5080. `vit` = ViT-Tiny (0.766), `r50` = ResNet-50 (0.746), `r18` = ResNet-18 (0.720), `eff` = EfficientNet-B0 (0.742), `mob` = MobileNet-V2 (0.718).

Seed	TBA→TPE	TBA (SA)	Random	TPE
0	vit	vit	vit	vit
1	vit	r50	r50	r50
2	vit	r18	vit	r50
3	vit	mob	vit	vit
4	eff	vit	vit	r50
5	vit	vit	vit	r50
6	vit	vit	vit	eff
7	vit	vit	vit	r50
8	eff	eff	r50	r50
9	vit	vit	vit	vit
vit rate	8/10	6/10	8/10	3/10

largest on H100 and RTX 5080; on A100 and T4, the methods are comparable.

The hybrid achieves lower waste than TPE on four of five GPUs. The hybrid wastes 28–42% of budget compared to TPE's 32–38%. On the H100, A100, L4, and T4, the hybrid's waste is lower than or equal to TPE's. On the RTX 5080, the hybrid's waste is higher (42% vs 36%) because it explores more broadly—but this broader exploration is precisely what drives its superior discovery rate (8/10 vs 3/10).

The hybrid's advantage is most visible on the H100 and RTX 5080. On the H100, the fastest GPU, the hybrid achieves perfect 5/5 discovery with only 28% waste—matching random search's discovery while wasting less than half the budget. On the RTX 5080, with 10 seeds, the hybrid's advantage is most visible: 8/10 vs 3/10 for TPE. The H100 result may seem surprising—why does a fast GPU with many feasible configs still show a gap? The answer is that TPE's failure mode is *model selection*, not latency: in seeds 2 and 4, TPE locks onto `resnet50` early and never evaluates `vit_tiny`, regardless of hardware speed.

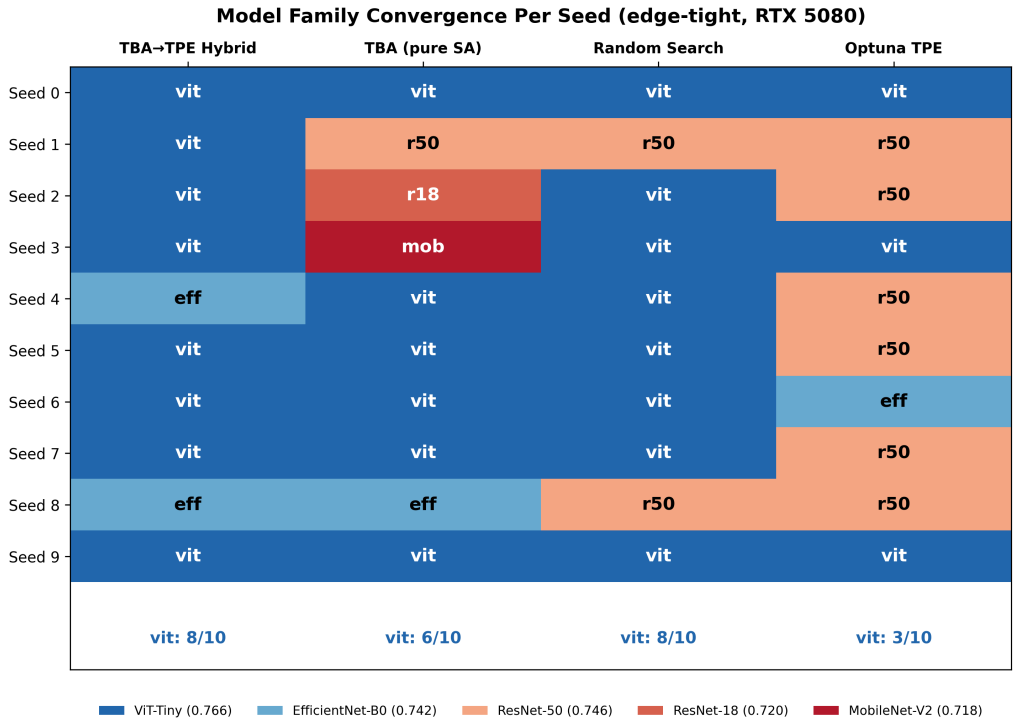

Figure 4: Model family convergence per seed under edge-tight constraints on RTX 5080. Each cell shows which model family each optimizer converged to. TPE locks onto `resnet50` in 7 of 10 seeds, while the hybrid discovers `vit_tiny` in 8 of 10 seeds.

Table 8: Seeds discovering `vit_tiny` under edge-tight constraints across five GPU targets. RTX 5080 uses 10 seeds; all others use 5 seeds.

Optimizer	H100	A100	RTX 5080	L4	T4
TBA→TPE Hybrid	**5/5**	**4/5**	**8/10**	**4/5**	3/5
TBA (pure SA)	3/5	2/5	6/10	1/5	2/5
Random Search	**5/5**	**5/5**	8/10	**4/5**	**4/5**
Optuna TPE	3/5	**4/5**	3/10	3/5	3/5

The T4 confirms the v2 improvements. In an earlier codebase without trial timeouts and subspace blacklisting, the hybrid discovered `vit_tiny` in only 2/5 T4 seeds, below TPE's 3/5. The exploration phase became trapped in `int8_dynamic` variants that consumed 500ms+ per trial. With the v2 improvements, the hybrid now matches TPE at 3/5 on T4 while reducing waste (36% vs 38%). Early stopping saved 7–47 seconds per seed, and subspace blacklisting prevented repeated proposals of `int8_dynamic` after consecutive failures. The T4 remains the most hostile environment, but it is no longer a failure case for the hybrid.

9.5 The Role of Random Search

An important observation is that random search matches the hybrid's `vit_tiny` discovery rate (8/10) while wasting 74% of budget. This is diagnostically useful. It confirms that `vit_tiny` is not hidden in an obscure corner of the space—the problem is not reachability but *search dynamics*. TPE's model-guided policy steers away from that family after committing early to `resnet50`. The hybrid preserves much of the structural coverage of broad random exploration while avoiding its extreme inefficiency.

Table 9: Mean accuracy and wasted budget fraction under edge-tight constraints across five GPU targets.

Optimizer	H100		A100		RTX 5080		L4		T4	
	Acc	W	Acc	W	Acc	W	Acc	W	Acc	W
TBA→TPE	.766	28%	.762	32%	.761	42%	.761	32%	.758	36%
TBA (SA)	.752	25%	.743	31%	.752	30%	.738	27%	.739	33%
Random	.766	63%	.766	66%	.762	74%	.762	67%	.760	72%
TPE	.758	32%	.762	33%	.752	36%	.758	34%	.758	38%

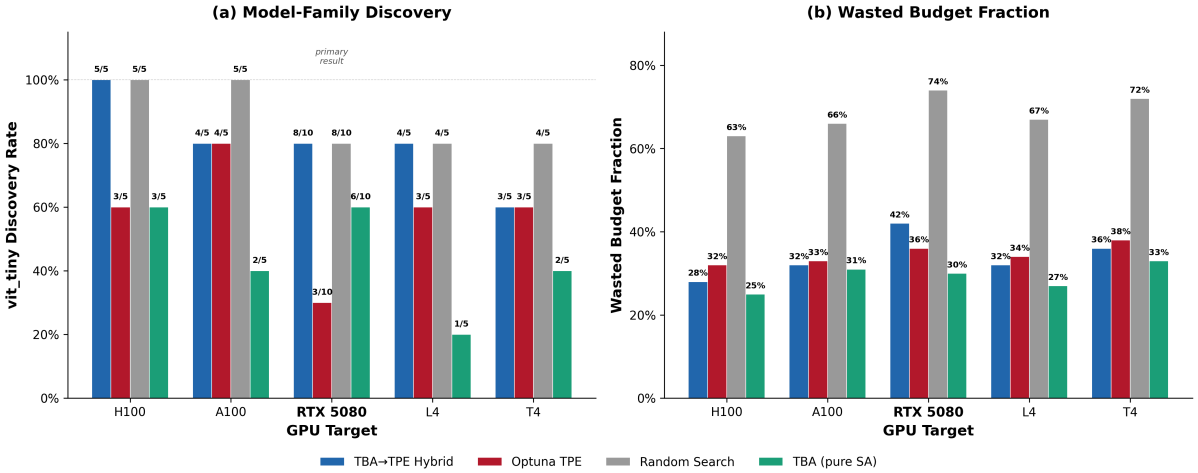

Figure 5: Discovery rate and wasted budget fraction across five GPU targets. The hybrid achieves high discovery with moderate waste, while TPE achieves lower waste but misses the best model family in many seeds. RTX 5080 uses 10 seeds; the other GPUs use 5 seeds. No other optimizer occupies the high-discovery, low-waste quadrant.

10 Analysis

10.1 Why Does TPE Miss the Best Model?

TPE's failure to discover `vit_tiny` in 7/10 RTX 5080 seeds and 2/5 H100 seeds can be explained by the interaction between crash rate and exploration depth. TPE uses an initial random design before switching to model-guided sampling. In a space where 65–80% of configurations crash or violate constraints, those initial trials may yield only 2–4 successful evaluations, potentially all from the same model family. Once the density estimator is fitted to this narrow sample, exploitation reinforces the same family rather than expanding structural coverage.

Using the formula from Remark 4.1 with $K = 5$, $\rho_{\text{crash}} = 0.6$, and $n_0 = 10$: $\mathbb{P} \approx 0.57$. This predicts that roughly 4–6 out of 10 seeds would discover any particular family, which is directionally consistent with the observed TPE discovery rates across hardware.

By contrast, the hybrid uses Phase 1 to widen structural coverage before handing off to TPE. This does not guarantee discovery of every promising family, but it raises the probability that the best family enters the candidate set prior to exploitation.

10.2 The Trade-Off Between Discovery and Waste

Across all five GPUs, the hybrid consistently occupies a distinct position in the discovery–waste trade-off:

- Random search achieves high discovery but extreme waste (63–74%).

- TPE achieves lower waste (32–38%) but misses the best model family in many seeds.
- Pure SA achieves the lowest waste (25–33%) but often converges to the wrong model family (resnet18, mobilenet_v2).
- The hybrid achieves high discovery with moderate waste (28–42%)—the best overall trade-off.

No other optimizer achieves both high discovery *and* low waste simultaneously. This is the practical value of the feasible-first exploration design.

10.3 Exploration–Exploitation Boundary

The multi-hardware experiments suggest that the hybrid's advantage is present across all hostility levels, though its magnitude varies. On fast GPUs (H100), the hybrid's broader Phase 1 exploration finds `vit_tiny` in every seed because the feasible region is large (batch sizes 1–32 all meet the 20ms constraint). On slow GPUs (T4), the feasible region is narrow and exploration traps are more likely, but trial timeouts and subspace blacklisting mitigate the worst cases. The hybrid's advantage is most visible on mid-range hardware (RTX 5080, L4) where the space is hostile enough that TPE struggles but not so hostile that even deliberate exploration becomes trapped.

11 Discussion

The results support the paper's central claim with appropriate nuance. The hybrid is not universally better than TPE on every metric. Rather, it changes the exploration–exploitation trade-off in settings where the early trial budget is too small, and the invalidity rate too high, for cold-start model-guided search to form a sufficiently broad view of the space.

H1 (structural coverage) is supported by the discovery-rate analysis: the hybrid's structural mutations during Phase 1 produce broader model-family coverage than TPE's cold-start random sampling in crash-heavy spaces. The effect is present across all five GPUs, with the hybrid matching or exceeding TPE's discovery rate on every hardware target, though the margin varies from substantial (RTX 5080: 8/10 vs 3/10) to negligible (T4: 3/5 vs 3/5).

H2 (invalidity changes the trade-off) is supported by the contrasting results across hardware targets: the hybrid's discovery advantage is largest on hardware where the crash-and-infeasibility rate is high enough to impair TPE's initial sampling (RTX 5080: 8/10 vs 3/10) but present even on the fastest hardware (H100: 5/5 vs 3/5).

H3 (warm-start helps) is supported by the ablation: TBA→TPE achieves -2.05 ± 0.71 versus TBA pure SA's -2.93 ± 2.37 at $B = 30$, with both better mean and substantially lower variance. The multi-hardware discovery rates further confirm this: pure SA discovers `vit_tiny` at lower rates than the hybrid on all tested GPUs (e.g., 3/5 vs 5/5 on H100, 1/5 vs 4/5 on L4, 6/10 vs 8/10 on RTX 5080).

A secondary implication concerns evaluation metrics. In deployment optimization, best feasible objective alone is insufficient. The hybrid's advantage manifests primarily in budget efficiency and discovery consistency, not in mean objective superiority. Trial waste is operationally meaningful because each evaluation may require model export, runtime compilation, repeated timing, and memory measurement. A method that obtains comparable feasible performance with substantially lower waste can be preferable in practice.

12 Limitations and Threats to Validity

Several factors limit the scope of the present conclusions.

Hardware coverage. While multi-hardware experiments span five GPU targets from datacenter (H100, A100) to edge (T4) and laptop (RTX 5080), all experiments use the same software stack and model set. Backend behavior and quantization support may differ on non-NVIDIA hardware or with different compiler versions.

Single deployment domain. The benchmark focuses on image classification deployment with a single scenario (edge-tight). Extensions to NLP serving, speech inference, multi-model pipelines [Zaharia et al., 2024], and additional constraint profiles remain future work.

Benchmark scale. The deployment space contains roughly 2000 structural combinations with five model families. Larger spaces with more model families may amplify the premature exploitation effect, but they may also challenge the exploration capacity of Phase 1.

Statistical scope. Ten seeds are used for the primary RTX 5080 experiments and five seeds for multi-hardware experiments. Larger seed counts would strengthen the conclusions, particularly for the T4 and L4 where differences of 1 seed (e.g., 3/5 vs 4/5) are not statistically significant.

Aggregate accuracy parity. The hybrid's edge-tight mean accuracy (0.761) is essentially tied with random search (0.762). Its advantage manifests primarily in discovery behavior and budget efficiency rather than large gains in average objective.

Residual exploration traps. While trial timeouts and subspace blacklisting substantially mitigate the T4 failure mode (improving from 2/5 to 3/5), the hybrid still does not achieve the same discovery rate on T4 as on faster hardware. On the A100, the hybrid also missed `vit_tiny` in seed 3 due to the SA phase becoming trapped in `int8_dynamic` variants that were slow but not slow enough to trigger the 5× early-stopping threshold.

Synthetic–real gap. The synthetic benchmarks reproduce structural invalidity and hidden crash regions but do not replicate every detail of real deployment failures.

13 Conclusion

This paper studied deployment optimization as a budget-constrained search problem in a partially invalid hierarchical space. The main result is not that simulated annealing outperforms mature model-guided optimizers in general. Instead, the evidence supports a narrower and more useful conclusion: when invalidity is high and budgets are small, *the quality of early exploration determines the effectiveness of later exploitation.* Warm-starting TPE with feasible-first exploration improves model-family discovery under tight constraints and reduces wasted budget relative to cold-start TPE.

Multi-hardware experiments across five GPU targets confirm that the hybrid never underperforms TPE on discovery, with the clearest gains on H100 (5/5 vs 3/5) and RTX 5080 (8/10 vs 3/10). On A100 and L4, the hybrid matches or slightly exceeds TPE (4/5 vs 4/5 and 4/5 vs 3/5, respectively). On the most hostile hardware, after the addition of trial timeouts and subspace blacklisting, the hybrid matches TPE with lower waste (T4: 3/5 vs 3/5, 36% vs 38% waste).

Three directions appear especially promising for future work. First, the handoff criterion could be made adaptive to observed structural diversity rather than simple counts, requiring minimum model-family coverage before initiating TPE. Second, hardware-transfer experiments could test whether feasibility maps learned on one device reduce search cost on another. Third,

the benchmark can be extended beyond image classification to include additional deployment scenarios, model families, and constraint profiles.

The benchmark suite and optimizer implementation are available at https://github.com/Chrislysen/Constrained-ML-Deployment.

A Additional Method Detail

A.1 Adaptive Structural Mutation Schedule

The structural mutation probability is decayed linearly:

$$p_s(n) = 0.5 - 0.35 \cdot \frac{n - n_{\text{init}}}{B - n_{\text{init}}}, \tag{17}$$

where n_{init} is the number of initial random trials.

A.2 Snap-Back and Elite Restart

The pure-TBA variant uses two stabilization mechanisms: (i) elite restart to the 2nd or 3rd best feasible state during early search, and (ii) decaying snap-back to the best feasible state after accepting a worse move. These mechanisms were retained because they improved the ablation metrics on Crashy Branin (Table 4).

A.3 Subspace Blacklisting Detail

The subspace tracker maintains a dictionary mapping each (categorical variable, value) pair to a consecutive failure count. On each trial outcome:

- **Crash or infeasible**: increment failure count for all categorical values in the configuration. If count reaches 3, blacklist the value.
- **Success (feasible)**: reset failure count to 0 for all categorical values in the configuration; remove any active blacklist.
- **Cooldown**: after 8 trials since blacklisting, automatically remove the blacklist regardless of outcomes.

The safety valve ensures that for each categorical variable, at least one value remains unblacklisted at all times.

B Computational Cost

Table 10: Wall-clock time for complete experimental evaluation.

Experiment	**Wall-clock**
Synthetic budget sweep (4 opt. × 7 budgets × 10 seeds)	∼15 min
Edge-tight deployment RTX 5080 (4 opt. × 10 seeds × 25 trials)	∼120 min
Multi-hardware edge-tight (4 opt. × 5 seeds × 25 trials × 4 GPUs)	∼6 hours
Total	∼8.5 hours

Primary experiments were conducted on a single NVIDIA RTX 5080 laptop GPU. Multi-hardware experiments were conducted on NVIDIA H100 80GB, A100 40GB, L4 24GB, and T4 16GB GPUs via Google Colab.

C Reproducibility Checklist

1. All experiments across all hardware targets use the same codebase (v2-improvements branch, commit `7ea93e2`).
2. Primary experiments used 10 random seeds (0–9); multi-hardware experiments used 5 seeds (0–4).
3. Crashed and early-stopped trials were logged rather than discarded.
4. The search-space specification, benchmark definitions, and experiment scripts are available in the public repository.
5. The reported tables are generated from saved JSONL result files.
6. Conditional parameters are exposed identically to all optimizers.
7. Trial timeouts are applied at the benchmark level and affect all optimizers equally.
8. Multi-hardware experiments were conducted in single clean sessions per GPU to avoid cross-session variability.

References

Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S. Gulavani, Ramachandran Ramjee, and Alexey Tumanov. Vidur: A large-scale simulation framework for LLM inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *KDD*, 2019.

Nikhil H. Awad, Neeratyoy Mallik, and Frank Hutter. DEHB: Evolutionary Hyperband for scalable, robust and efficient hyperparameter optimization. In *IJCAI*, 2021.

James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In *NeurIPS*, 2011.

James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.

Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Lydia Wang, Yaoze Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: An automated end-to-end optimizing compiler for deep learning. In *OSDI*, 2018.

Katharina Eggensperger, Philipp Müller, Neeratyoy Mallik, et al. HPOBench: A collection of reproducible multi-fidelity benchmark problems for HPO. In *NeurIPS Datasets and Benchmarks Track*, 2021.

Stefan Falkner, Aaron Klein, and Frank Hutter. BOHB: Robust and efficient hyperparameter optimization at scale. In *ICML*, 2018.

Jacob R. Gardner, Matt J. Kusner, Zhixiang Xu, Kilian Q. Weinberger, and John P. Cunningham. Bayesian optimization with inequality constraints. In *ICML*, 2014.

Roman Garnett. *Bayesian Optimization*. Cambridge University Press, 2023.

Michael A. Gelbart, Jasper Snoek, and Ryan P. Adams. Bayesian optimization with unknown constraints. In *UAI*, 2014.

Nikolaus Hansen and Andreas Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2):159–195, 2001.

José Miguel Hernández-Lobato, Michael Gelbart, Ryan Adams, Matthew Hoffman, and Zoubin Ghahramani. A general framework for constrained Bayesian optimization using information-based search. *Journal of Machine Learning Research*, 17(160):1–53, 2016.

Jeremy Howard. Imagenette: A smaller subset of 10 easily classified classes from ImageNet. https://github.com/fastai/imagenette, 2020.

Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren (Eds.) *Automated Machine Learning: Methods, Systems, Challenges*. Springer, 2019.

Shawn Hymel, Colby Banbury, Daniel Situnayake, et al. Edge Impulse: An MLOps platform for tiny machine learning. In *Proceedings of Machine Learning and Systems (MLSys)*, 2023.

Lester Ingber. Very fast simulated re-annealing. *Mathematical and Computer Modelling*, 12(8):967–973, 1989.

Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52, 2017.

Marius Lindauer, Katharina Eggensperger, Matthias Feurer, et al. SMAC3: A versatile Bayesian optimization package for hyperparameter optimization. *Journal of Machine Learning Research*, 23(54):1–9, 2022.

Florian Pfisterer, Lennart Schneider, Julia Moosbauer, Martin Binder, and Bernd Bischl. YAHPO Gym: An efficient multi-objective multi-fidelity benchmark for hyperparameter optimization. In *AutoML Conference*, 2022.

Vijay Janapa Reddi et al. MLPerf inference benchmark. In *ISCA*, 2020.

Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. Practical Bayesian optimization of machine learning algorithms. In *NeurIPS*, 2012.

Yunsheng Tian, Ane Zuniga, Xinwei Zhang, Johannes P. Dürholt, Payel Das, Jie Chen, Wojciech Matusik, and Mina Konaković Luković. Boundary exploration for Bayesian optimization with unknown physical constraints. In *ICML*, 2024.

Shuhei Watanabe and Frank Hutter. c-TPE: Tree-structured Parzen estimator with inequality constraints for expensive hyperparameter optimization. In *IJCAI*, 2023.

Matei Zaharia, Omar Khattab, Lingjiao Chen, et al. The shift to compound AI systems. *Berkeley AI Research Blog*, 2024.

Lianmin Zheng, Chengfan Jia, Minmin Sun, et al. Ansor: Generating high-performance tensor programs for deep learning. In *OSDI*, 2020.

Algorithm 1 TBA→TPE Hybrid Optimizer

Require: Search space $\mathcal{X}$, constraints $\{(g_j, c_j)\}$, objective f, budget B

$\mathcal{H} \leftarrow \emptyset$, $T \leftarrow T_0$, phase $\leftarrow$ `feasibility`
Initialize subspace tracker $\mathcal{T}$ and early-stopping threshold $\tau = 5 \cdot c_{\text{latency}}$
Sample n_{init} random configurations; evaluate and add to $\mathcal{H}$
$\boldsymbol{x}_{\text{curr}} \leftarrow$ best feasible in $\mathcal{H}$ (or least-violating if none feasible)
for $n = n_{\text{init}} + 1$ to adaptive handoff **do**
  Compute structural mutation probability $p_s(n)$
  Mask blacklisted categorical values from $\mathcal{T}$ when proposing
  if elite restart condition met and $n < 0.7B$ **then**
    $\boldsymbol{x}' \leftarrow$ 2nd or 3rd best feasible from $\mathcal{H}$
  else
    $\boldsymbol{x}' \leftarrow \text{ProposeNeighbor}(\boldsymbol{x}_{\text{curr}}, T, p_s(n))$
  end if
  Evaluate $\boldsymbol{x}'$ with early stopping: abort if latency $> \tau$ after warmup
  Add $(\boldsymbol{x}', r')$ to $\mathcal{H}$; update $\mathcal{T}$ with outcome
  if r' indicates crash or early stop **then**
    Update T as a non-improving step; keep $\boldsymbol{x}_{\text{curr}}$
  else if phase = `feasibility` **then**
    Apply violation-based Metropolis update
  else
    Apply objective-based Metropolis update with hard feasibility filter
  end if
  if HandoffReady($\mathcal{H}$) **then**
    break
  end if
end for
Create Optuna study with constrained TPE sampler [Watanabe and Hutter, 2023]
for each $(\boldsymbol{x}_i, r_i) \in \mathcal{H}$ **do**
  Inject as completed trial with objective value and constraint metadata
end for
for $n = |\mathcal{H}| + 1$ to B **do**
  Request next configuration from TPE (full search space, no blacklisting)
  Evaluate with early stopping and report result to the study
end for
return best feasible configuration observed